

RAPPORT TECHNIQUE GLOBAL ET COMPLET : APPLICATION « JEU » — LES CARTOGRAPHES DU MÉTRO

INTRODUCTION GÉNÉRALE

Ce document constitue le descriptif technique complet de l'**application Jeu**, qui permet de jouer une partie des *Cartographes du Métro* sur un plateau préalablement conçu dans l'application Conception.

L'architecture logicielle repose sur le patron de conception **MVC (Modèle-Vue-Contrôleur)**, implémenté à l'aide des bibliothèques graphiques **Java Swing et AWT**. L'objectif est d'isoler de manière étanche la **logique métier** (règles du jeu, pioche, calcul du score, graphe du plateau) de ses **représentations visuelles** (l'IHM). Le **Contrôleur** fait office de passerelle unique : la vue n'accède jamais directement aux objets du modèle, elle passe toujours par lui, ce qui garantit l'intégrité des structures de données.

Le code est organisé en trois paquets : - `plateau` : le **Contrôleur**, classe pivot de l'application ; - `plateau.ihm` : toutes les **fenêtres et panneaux** Swing ; - `plateau.metier` : le **cœur logique** autonome, sans aucune dépendance graphique.

La partie se déroule en **mode local** : une fenêtre par joueur affiche son propre plateau, et une **bande d'informations centrale** commune affiche la carte piochée, l'état des joueurs et les commandes.

1. LE PACKAGE PRINCIPAL (`plateau`)

`Contrôleur.java`

Centre décisionnel et communicationnel de l'architecture. Il détient l'état courant de la partie et distribue les ordres de mise à jour entre le modèle et les fenêtres.

Attributs

- `ihm (FrameAccueil)` : fenêtre d'accueil, premier écran instancié au lancement.
- `metier (Plateau)` : instance du modèle hébergeant le plateau courant (cases, stations, départs, graphe).
- `partie (Partie)` : instance gérant le déroulement d'une partie locale (manches, tours, pioche, scores).
- `framesJoueurs (FrameJoueur[])` : tableau des fenêtres de jeu, une par joueur.
- `frameInfos (FrameInfos)` : référence vers la bande d'informations centrale.
- `resultatsAffiches (boolean)` : verrou empêchant d'afficher deux fois l'écran final.
- `COULEURS_JOUEURS (static final Color[])` : palette fixe de 4 couleurs (rouge, bleu, vert, orange) attribuées aux joueurs ; elles **tournent** à chaque manche.
- `imagesStations (Image[])` : cache des images de stations (indices 1 à 10).
- `imagesCartes (Image[][])` : cache des images de cartes, dimension `[numéro][0=claire, 1=foncée]`.

Descriptif granulaire des méthodes

- `Contrôleur()` (*constructeur*) : crée un `Plateau` 10×10 par défaut puis instancie et affiche la fenêtre d'accueil `FrameAccueil`.

- **chargerPlateau(File fichier)** : délègue la lecture au `ChargeurPlateau` ; si le plateau renvoyé n'est pas nul, il remplace le modèle courant et renvoie `true`.
- **getSauvegardes()** : parcourt plusieurs chemins candidats vers le dossier de sauvegardes de l'application Conception et renvoie la liste des fichiers `.txt` trouvés sous forme de tableau `File[]`.
- **plateauEstJouable()** : prédicat de validité ; renvoie `true` uniquement si le plateau contient **au moins un départ et au moins une station**.
- **lancerPartieLocale(int nbManches)** : instancie la `Partie`, crée la bande centrale `FrameInfos`, puis une `FrameJoueur` par joueur détecté, et appelle `placerFenetres()`.
- **placerFenetres()** : calcule la disposition à l'écran ; place la bande d'infos au centre et répartit les plateaux des joueurs de part et d'autre, en s'adaptant aux dimensions de l'écran (`Toolkit.getScreenSize`).
- **jouerCoup(int numeroJoueur, int numCase)** : transmet la tentative de pose à la `Partie` et renvoie son verdict de validité.
- **passerTour(int numeroJoueur)** : ordonne à la `Partie` de faire passer le tour du joueur.
- **rafraichirTout()** : rafraîchit toutes les fenêtres puis gère la suite logique : si une manche vient de finir, affiche les scores et enchaîne la manche suivante ; si la partie est terminée, affiche l'écran de résultats (une seule fois).
- **rafraichirFrames()** : commande le rafraîchissement visuel de la bande d'infos et de chaque fenêtre joueur.
- **afficherScoresManche()** : construit un texte récapitulatif du score de la manche écoulée et l'affiche dans une boîte de dialogue `JOptionPane`.
- **isPartieTerminee(), aJoue(int), estSonTour(int), getJoueurCourant(), getCarteCourante(), getNbFoncesRestantes(), getCaseDepart(int), getNbJoueurs(), getNbManches(), getNumeroManche(), getJoueurs()** : accesseurs qui **délèguent** à la `Partie`, avec garde contre l'absence de partie (`null`).
- **getCasesValides(int numeroJoueur)** : renvoie la liste des cases jouables pour un joueur (utilisée pour la surbrillance jaune).
- **getCheminJoueur(int numeroJoueur)** : renvoie la suite ordonnée des stations du réseau d'un joueur (pour tracer sa ligne).
- **getCouleurJoueur(int numeroJoueur, int numeroManche)** : calcule l'indice $((\text{joueur}-1)+(\text{manche}-1)) \% 4$ pour renvoyer la couleur du joueur ; c'est cette formule qui fait **tourner les couleurs** d'une manche à l'autre.
- **getCouleurJoueur(int numeroJoueur)** : surcharge utilisant la manche courante.
- **getGagnants()** : détermine le ou les gagnants — meilleur **score total** ; en cas d'égalité, départage par la **meilleure manche** ; sinon victoire **partagée**.
- **getLargeur(), getHauteur(), getArrondissement(int), getStation(int)** : accesseurs en lecture seule sur le plateau.
- **getImageStation(int numero)** : renvoie l'image d'une station, avec **mise en cache paresseuse** (chargée une seule fois).
- **getImageCarte(Carte carte)** : renvoie l'image d'une carte (le joker porte le numéro 7), selon qu'elle est claire ou foncée, avec mise en cache.
- **getImageFond() / getImageFond2()** : renvoient les chemins absolus des images de fond des écrans d'accueil et de chargement.
- **chercherImage(String nom)** : utilitaire cherchant une image dans plusieurs emplacements possibles.

- `main(String[] a)` : point d'entrée ; instancie simplement un `Contrôleur`.

2. LE PACKAGE INTERFACE GRAPHIQUE (`plateau.ihm`)

`FrameAccueil.java`

Fenêtre d'accueil (724×965) servant d'écran au panneau d'accueil. Elle fixe le titre, ajoute le `PanelAccueil`, configure la fermeture (`EXIT_ON_CLOSE`), se centre à l'écran et s'affiche.

`PanelAccueil.java`

Panneau d'arrière-plan de l'accueil, affichant une image de fond et les deux boutons principaux. - **`PanelAccueil(...)` (constructeur)** : charge l'image de fond, fixe une taille préférée (800×600), crée les boutons « **Jouer** » et « **Règles** », les centre via un `GridBagLayout` enveloppant un `GridLayout(2,1)`, puis branche les écouteurs. - **`actionPerformed(ActionEvent e)`** : sur « Jouer », ouvre l'écran de chargement et masque l'accueil ; sur « Règles », recherche le fichier PDF des règles et l'ouvre avec le lecteur système par défaut (`Desktop.open`), sinon affiche un message console. - **`paintComponent(Graphics g)`** : redessine et étire l'image de fond sur toute la surface du panneau.

`FrameChargement.java`

Fenêtre de sélection (`FlowLayout`, `pack()`) accueillant le panneau de chargement ; centrée à l'écran.

`PanelChargement.java`

Écran de sélection d'un plateau et des paramètres de partie. - **Attributs** : `comboFichiers` (liste des sauvegardes), boutons radio `rbLocal/rbMulti` (mode de jeu), champ `txtManches` (nombre de manches), boutons `btnCharger/btnRetour`, label `lblStatut` (messages d'erreur en rouge), image de fond. - **`PanelChargement(...)` (constructeur)** : structuré en trois sections (**création, positionnement, activation**) ; remplit la combo avec `getSauvegardes()`, regroupe les boutons radio dans un `ButtonGroup`, et désactive « Charger » si aucune sauvegarde n'existe. - **`actionPerformed(ActionEvent e)`** : sur « Charger », charge le plateau, vérifie qu'il est **jouable** (sinon « Plateau non finalisé »), puis selon le mode : multijoueur -> message « à venir » (non implémenté) ; local -> lit et **valide le nombre de manches** (numérique, ≥ 1) avant de lancer la partie. Sur « Retour », revient à l'accueil. - **`paintComponent(Graphics g)`** : dessine l'image de fond puis superpose un **voile noir translucide** ($\alpha=130$) pour la lisibilité du texte blanc.

`FrameInfos.java` / `PanelInfos.java`

Longue **bande verticale centrale** commune à tous les joueurs. - **`FrameInfos`** : conteneur de la bande ; sa méthode `rafraichir()` délègue au panneau. - **`PanelInfos`** — attributs : `panelCarte` (image de la carte commune), `lblManche` (numéro de manche), `lblPioche` (compteur de cartes foncées restantes), tableaux `lblStatutJoueur[]` et `btnPasserJoueur[]` (état et bouton « Passer » de chaque joueur), bouton `btnQuitter`. - **`PanelInfos(...)` (constructeur)** : empile verticalement (`BoxLayout`) le titre, la manche, la carte commune, le compteur de cartes foncées, puis pour chaque joueur son statut et son bouton « Passer », et enfin le bouton « Quitter ». - **`actionPerformed(...)`** : « Quitter » termine le programme ; un bouton « Passer (Joueur n) » fait passer le tour de ce joueur puis rafraîchit tout. - **`rafraichir()`** : met à jour le numéro de manche, repeint la carte, actualise le compteur de cartes foncées, et pour chaque joueur affiche l'état approprié (**à jouer / a déjà joué / en attente / terminé**) en activant ou désactivant son bouton « Passer ».

FrameJoueur.java / PanelJoueur.java

Fenêtre de jeu d'un joueur, ne contenant que son plateau. - **FrameJoueur** : porte le titre « Joueur n » ; `rafraichir()` délègue au panneau. - **PanelJoueur** : contient un `PanelGrille` (la grille). `caseCliquee(int index)` : si c'est bien le tour de ce joueur, transmet la pose au contrôleur puis **rafraîchit toutes les fenêtres**. `rafraichir()` repeint la grille.

PanelGrille.java

Plateau d'un joueur, capturant les clics et dessinant tout le rendu. - `mousePressed(MouseEvent e)` : convertit la position en pixels du clic en **indice de case** (division par la taille des cellules) et le transmet au `PanelJoueur`. - `construireGrille()` : (re)dimensionne le panneau selon les dimensions du plateau chargé. - `paintComponent(Graphics g)` : dessine successivement le fond clair, les **zones d'arrondissement** colorées en translucide, la **ligne** du réseau du joueur (dans l'ordre de pose, à sa couleur), les **stations** (images), la **surbrillance jaune** des coups valides, et enfin un **anneau** de couleur autour de la case de départ du joueur.

Géométrie : centrage du tracé du réseau. Pour relier deux stations consécutives, le code ne part pas du coin de la cellule mais de son **centre** : il calcule $x = (case \% largeur) * cellw + cellw/2$ et $y = (case / largeur) * cellh + cellh/2$. Ce barycentre garantit des lignes nettes qui s'adaptent dynamiquement si la fenêtre est redimensionnée.

PanelCarte.java

Affiche l'image de la **carte commune** courante (claire, foncée, ou joker = numéro 7). Si la pioche est épuisée, il n'affiche rien.

FrameResultats.java / PanelResultats.java

Écran final de la partie. - `PanelResultats(...)` (*constructeur*) : construit le **tableau des scores** (une ligne par joueur, une colonne par manche M1...Mn plus le **Total**) à l'aide de méthodes utilitaires `entete()` et `cellule()`, puis affiche le **gagnant** (ou la mention « Égalité, victoire partagée ») et le bouton « Quitter ». - `actionPerformed(...)` : « Quitter » termine le programme.

3. LE PACKAGE LOGIQUE MÉTIER (`plateau.metier`)

Ce paquet est **autonome** : il ne contient aucune référence à Swing/AWT et se consacre uniquement aux structures de données et aux règles du jeu.

Plateau.java

Classe maîtresse du modèle. Elle mémorise les dimensions et trois tableaux d'entiers indexés par numéro de case : `tabArrondissements`, `tabStations`, `tabDeparts`, plus le graphe d'adjacence. - **Méthodes** : `getLargeur/getHauteur`, `getGraphe`, et les couples accesseur/mutateur `getArrondissement/affecterArrondissement`, `getStation/affecterStation`, `getDepart/affecterDepart` (tous bornés pour éviter les débordements de tableau).

GraphePlateau.java

Modélise le réseau de liaisons entre stations par une **matrice d'adjacence** booléenne `matriceArete[][]`. - `aArete(i, j)` : prédicat indiquant si une arête relie deux cases. -

ajouterArete(i, j) : pose l'arête de manière **symétrique** ([i][j] et [j][i]), car le graphe est **non orienté**. - **vider()** : réinitialise toute la matrice. - **genererAretesAuto(Plateau)** : génère automatiquement les liaisons (voir encart).

Analyse spatiale : génération des arêtes (genererAretesAuto). L'algorithme procède par **lancer de rayon** (*ray-casting*). Pour chaque station, il explore 4 directions (Est, Sud-Est, Sud, Sud-Ouest — les 4 autres étant couvertes par symétrie) et **avance case par case** jusqu'à rencontrer la **première** station, à laquelle il se relie avant de s'arrêter (break). Deux stations sont donc voisines si elles sont sur le même axe et qu'**aucune autre station ne s'intercale** entre elles.

Carte.java

Classe représentant une carte de la pioche. Champs : `typeStation` (1..N, ou JOKER = 0) et `estFoncee` (booléen). Méthodes : `getTypeStation`, `estFoncee`, `estJoker`.

Pioche.java

Modélise la **pioche commune** à tous les joueurs. - **Pioche(int nbStations)** (*constructeur*) : construit le paquet — une carte **foncée** par type de station plus un **joker foncé**, puis autant de cartes **claires** et un **joker clair**. Le compteur `nbFonceesRestantes` est initialisé. - **melanger()** : brassage par l'algorithme de **Fisher-Yates** (voir encart). - **piocher()** : retire et renvoie la carte du dessus ; décrémente le compteur de foncées si la carte tirée est foncée ; renvoie `null` si la pioche est vide. - **estTerminee()** : indique si toutes les cartes **foncées** ont été tirées (condition de fin de manche). - **estVide()**, **getNbCartes()**, **getNbFonceesRestantes()** : accesseurs d'état du paquet.

Analyse : le brassage de Fisher-Yates (melanger). La méthode parcourt la liste depuis la fin et échange chaque carte avec une carte tirée aléatoirement parmi celles qui restent (un seul passage, complexité linéaire **O(n)**). C'est la méthode mathématiquement juste : chacune des **N!** permutations possibles du paquet a exactement la même probabilité d'apparaître, sans biais.

Joueur.java

Représente un joueur : son `reseau` (sa ligne), un `drapeau aJoue` (verrou d'une action par carte), le tableau `scoresManches[]` et le `scoreTotal`. - **Méthodes** : `commencerReseau(caseDepart)` (réinitialise le réseau au départ d'une manche), `ajouterScore`, `getScoreManche`, `getMeilleurScoreManche` (sert au départage), `getReseau`, `aJoue/setAJoue`, `getScoreTotal`.

ReseauJoueur.java

Modélise la **ligne de métro** d'un joueur : une **liste ordonnée** (`ArrayList<Integer>`) de stations. - **ajouterStation(numCase)** : prolonge la ligne par la **fin**. - **ajouterDebut(numCase)** : prolonge la ligne par le **début** (insertion en tête). - **contient**, **reinitialiser**, **getPremiereStation**, **getDerniereStation** : la première et la dernière station constituent les **deux extrémités** par lesquelles la ligne peut grandir.

ValideurMouvement.java

Vérifie qu'un joueur peut poser la carte courante sur une case donnée. - **estValide(numCase, joueur, carte, plateau)** : applique les **4 règles dans l'ordre** (voir encart) et renvoie un booléen. - **getCasesValides(joueur, carte, plateau)** : parcourt toutes les cases et renvoie la liste de celles qui passent la validation (sert à la surbrillance jaune).

Analyse : les 4 règles de validation (estValide). Dans l'ordre, le coup est refusé si : **(1)** la case ne porte pas de station ; **(2)** son type ne correspond pas à la carte (sauf joker) ; **(3)** la station est déjà

dans le réseau du joueur ; **(4)** elle n'est reliée par une **arête** à aucune des **deux extrémités** du tracé. L'ordre compte : chaque test sort immédiatement, et c'est pourquoi un test de type erroné « masque » les tests suivants.

Partie.java

Orchestre le déroulement d'une partie locale, en plusieurs manches, avec **pioche commune**. - **Attributs** : plateau, joueurs[], nbJoueurs/nbStations (détectés sur le plateau), casesDepart[], nbManches/numeroManche, pioche, carteCourante, joueurCourant, et les drapeaux entreManche/partieTerminee. - **Partie(plateau, nbManches)** (*constructeur*) : détecte le nombre de joueurs (max des départs) et de stations (max des types), repère la case de chaque départ, crée les joueurs, et démarre la première manche. - **getCaseDepart(numeroJoueur)** : renvoie la case de départ d'un joueur pour la manche courante — les départs **tournent** à chaque manche $((\text{joueur}-1)+(\text{manche}-1)) \% \text{nbJoueurs})$. - **demarrerManche()** : replace chaque joueur sur son départ, recrée et mélange la pioche, révèle la première carte commune, et donne la main au joueur 1. - **jouerCoup(numeroJoueur, numCase)** : vérifie le tour et les 4 règles, puis **rattache** la nouvelle station à la **bonne extrémité** (fin via ajouterStation ou début via ajouterDebut selon l'arête disponible), verrouille le joueur et passe au suivant. - **passerTour(numeroJoueur)** : marque le joueur comme ayant joué sans poser, puis avance. - **avancerJoueur()** : donne la main au prochain joueur n'ayant pas joué ; quand tous ont joué la carte commune, révèle la carte suivante — ou termine la manche si toutes les foncees sont tirées. - **finManche()** : calcule le score de chaque joueur via CalculateurScore, puis passe en attente de la manche suivante (entreManche) ou termine la partie. - **continuerManche()** : enchaîne sur la manche suivante (plateau remis à vide, départs tournés). - **estSonTour, getCasesValides, estDansReseau, getCheminJoueur, aJoue** : interrogations d'état pour l'IHM. - **Getters** : getPlateau, getJoueurs, getNbJoueurs, getNbManches, getNumeroManche, getCarteCourante, isEntreManche, isPartieTerminee, getJoueurCourant, getNbCartesRestantes, getNbFonceesRestantes.

Analyse : rotation pour l'équité. À chaque manche, **les départs et les couleurs tournent** entre les joueurs (même formule modulo). Cela évite qu'un joueur soit avantagé par une position de départ favorable sur toute la partie.

CalculateurScore.java

Calcule le score d'un réseau en fin de manche.

Analyse : le calcul multiplicatif du score (calculer). Le barème n'est pas une simple somme : il vaut **(nombre de zones traversées) × (plus grand nombre de stations dans une seule zone)**. Une « zone » est un arrondissement. La méthode compte les stations du tracé par arrondissement, puis multiplie le **nombre de zones non vides** par le **maximum** atteint dans l'une d'elles. Cette formule pousse le joueur à un compromis stratégique entre **étendue** (traverser beaucoup de zones) et **densité** (concentrer ses stations).

UtilitaireJeu.java

Boîte à outils statique : getNomsStations() (noms thématiques des stations), getCouleurs() (20 couleurs d'arrondissements), getCheminImageCarte(numero, foncee) et getCheminImageStation(numero) (résolution des chemins d'images dans plusieurs emplacements).

ChargeurPlateau.java

Décodeur qui lit un fichier `.txt` tabulé par `;` et instancie un `Plateau`. - **`charger(File fichier)`** : valide le fichier puis lit l'en-tête `largeur;hauteur`, les lignes de cases `index;arrondissement;station;depart`, et les éventuelles lignes d'arêtes `source;destination`.

Analyse : robustesse de la lecture (`charger`). Le chargeur applique une **batterie de sécurités** et renvoie `null` au moindre problème : fichier absent/illisible, en-tête != 2 valeurs, dimensions hors bornes (`<=0` ou `>100`), indices non séquentiels, valeurs hors plage (arrondissement 0–20, station 0–10, départ 0–4), nombre de cases incorrect, ou valeurs non numériques (`NumberFormatException`). Si le fichier ne contient pas d'arêtes pré-enregistrées, elles sont **générées automatiquement** par le graphe.

Fin du rapport technique — Application Jeu.